Sonuç

Karmaşık bir sistemi ne yapacağını varsaymadan tasarlayamazsınız.

Diyagramlar problemi nasıl anladığınızı ve çözümünüzü nasıl ifade ettiğinizi temsil eder.

UML'de kullanabileceğiniz pek çok diyagram vardır, bunların yanında OCL ve meta model de mevcuttur.

Problemi ne kadar iyi anlarsanız, sistemin geliştirme geçmişinde o kadar az alt problemle karşılaşırsınız.

Nesneye Yönelik Analiz

Tasarıma Nasıl Başlarsınız?

Problemi çözmeniz için önce anlamanız gerekir. → Analiz

Nesneye Yönelik Analiz gerçekleştireceğiz.

Abbott ve Booch tarafından 1980'lerde geliştirildi.

Nesne analiz modeli yaratmak için, doğal dil ile tanımlanmış gerçek dünya nesnelerini modellemeye odaklanır.

UML sınıf diyagramları ile ifade edilir. Önemli olan, problem ile ilgili nesnelerdir.

Alternatifi fonksiyonel çözümlemedir. Çözümün sağlaması gereken fonksiyonlara odaklanır.

Neden Fonksiyon yerine Nesneler?

Yazılım sistemlerinin uzun yaşam ömürleri vardır ve bu süre boyunca evrimsel değişikliklere uğrarlar.

Özellikler eklendikçe, hatalar düzeltildikçe, kütüphaneler değiştikçe programın orijinal yapısı da kaybolur.

Bakım esnasında fonksiyonlar nesnelerden çok daha sık değişir.

Örnek: Bankacılık uygulaması.

- Faiz hesabı, İşlem ücreti hesaplama, vergi alma gibi FONKSİYONlar zaman içinde değişir.
- Hesaplar, Müşteriler, İşlemler gibi NESNEler ise değişmez.

Nesneler fonksiyonlardan daha kararlı yapılardır.

- Nesneler, daha sürdürülebilir tasarımlar sağlar.

Nesneye Yönelik Analiz ve Tasarım

Yapısal analiz ve tasarım, fonksiyon yönelimlidir.

Yapılması gereken hesaplamalar üzerine yoğunlaşırlar.

Hesaplamaların üzerinde işlediği veriler ise ikinci plandadır.

Nesneye yönelik analiz ve tasarım veri nesnelerini ilk planda ele alır.
- Özellikler ve veri tipleri önce modellenir.
- Sonra nesnelere davranışlar (fonksiyonlar) eklenir.

Nesneye Yönelik Analiz

Gerçekleştirilecek sistemin metin ifadesini (gereksinim dokümanları) alırız.
- Ya da oluştururuz.

İsim, fiil, sıfat gibi kelime tiplerini araştırırız (altlarını çizeriz)
- Sınıfları isimlerle
- Özellikleri sıfatlarla
- İşlemleri eylem bildiren fiillerle
- İlişkileri durum bildiren fiillerle oluştururuz.

Oluşan sınıf modeli müşteri ile gözden geçirilebilir.

Kolay gibi görünse de ancak düşünerek düzgün çözebileceğimiz tarafları da olacak.

Bir ağacın yapraklarını saymak
1. İsimlerden aday sınıflara

Ağaç: Döngü içermeyen çizge. Her düğümün sıfır ya da daha fazla alt düğümü ve en fazla bir üst düğümü olabilir.

Yöntem:

Sayılmamış ağaç parçalarını alıp boş yığına koyun.

Yaprakların sayacını sıfır yapın.

Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.

Ağaçta tek yaprak varsa, yaprakların sayacını arttırın ve ağacı atın.

İki tane alt ağaç varsa sol ve sağı ayırıp ikisini de yığına ekleyin.

Yığın boşaldığında, yaprak sayısını gösterin.

Bir ağacın yapraklarını saymak
1. İsimlerden aday sınıflara

Ağaç: Döngü içermeyen çizge. Her düğümün sıfır ya da daha fazla alt düğümü ve en fazla bir üst düğümü olabilir.

Yöntem:

Sayılmamış **ağaç parçalarını** alıp boş **yığına** koyun.

Yaprakların sayacını sıfır yapın.

Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.

Ağaçta tek **yaprak** varsa, yaprakların sayacını arttırın ve ağacı atın.

İki tane **alt ağaç** varsa sol ve sağı ayırıp ikisini de yığına ekleyin.

Yığın boşaldığında, yaprak **sayısını** gösterin.

Başımıza gelenler

Nesneye Yönelik Analize isimlerle başlayınca olanlar:

Bazı kelimeler tekrar eder.
- Ağaç, ağaç, ağaç

Bazı kelimelerin kökü ortaktır.

Bazı kelimeler anlamsal olarak birbirine çok yakındır ve aynı saklı konsepti içerir.
- Yaprak yapraklar

Bunları yoğunlaştırıp tek kopyasını ve kelime kökünü kullanacağız.
- Sayaç / Sayı ?

2. İsimlerimizden Aday Sınıflara

Yığın, Parçalar, Ağaç, AltAğaç, Sayı, Sayaç, Yaprak, Yapraklar, Sıfır

Temizleyerek aday sınıflarımızı elde edelim:

Yığın → Önemli bir kavram olarak görünüyor.

Parça ? Alt ağaç da olabilir?

Ağaç → bu da önemli

Sayaç → bu da önemli

Yaprak ? Özel bir ağaç isimlendirmesi olabilir mi?

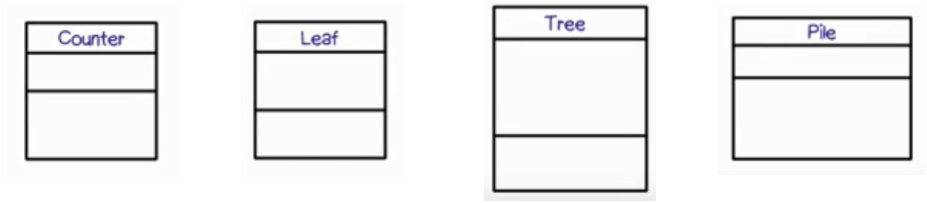
Düğüm? Analist probleme orjinal tanımda olmayan, hiç düşünülmemiş kavram ilave edebilir. Problemli, tutarsız ya da eksik kısımları ortaya çıkarabilir.

Sıfır? → Sayacın bir özelliğinin değeri olabilir. Tabii modelimizin bir parçası. Sonra unutmayın!

UML Kavramsal Sınıf Diyagramı

Aday sınıflardan gruplar oluşturulur ve analiz modeli Kavramsal sınıf diyagramı ile ifade edilir.

Sınıf: birbiri ile ilgili nesne grubunun tanımı



Analiz modeli için uyarılar

Tüm analizlerde olduğu gibi, vardığımız sonuçlar kesin değildir.

Süreçte ilerledikçe, problem hakkında daha fazla bilgi sahibi oldukça, modelimiz revizyona uğrar.

Analist, gereksinim belgelerindeki eksikleri gidererek doğru (müşterinin gerçekten istediği) modeli oluşturabilmelidir.

- Müşterilere geri dönmek gerekebilir.
- Çoğu zaman, sizin soru olarak getirdiğiniz şeyi düşünmemişlerdir bile.
- Sonunda size teşekkür ederler.

Süreç, analist ve müşteri arasında artımlı olarak devam eder ve ortak anlaşılmış bir noktada son bulur.

Bir ağacın yapraklarını saymak 3. Sıfatlardan özelliklere

Sıfatları, etkiledikleri isimlerle birlikte işaretleyelim:

Sayılmamış ağaç parçalarını alıp boş yığına koyun.
Yaprakları sayacını sıfır yapın.
Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.
Ağaçta tek yaprak varsa, yaprakların sayacını arttırın ve ağacı atın.
İki tane alt ağaç varsa sol ve sağı ayırıp ikisini de yığına ekleyin.
Yığın boşaldığında, yaprak sayısını gösterin.

Bir ağacın yapraklarını saymak 3. Sıfatlardan özelliklere

Sıfatları, etkiledikleri isimlerle birlikte işaretleyelim:

Sayılmamış ağaç parçalarını alıp **boş yığına** koyun.
Yaprakların sayacını sıfır yapın.
Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.
Ağaçta **tek yaprak** varsa, yaprakların sayacını arttırın ve ağacı atın.
İki tane alt **ağaç** varsa **sol** ve **sağı** ayırıp ikisini de yığına ekleyin.
Yığın boşaldığında, yaprak sayısını gösterin.

Bir ağacın yapraklarını saymak 3. Sıfatlardan özelliklere

Yığın: boş

Ağaç: iki, sol, sağ

Sayaç: yaprak

Yaprak: tek

Parça:

Her sıfat özellik olacak diye bir şart yok. İlgisizleri elemeliyiz.

Edatlardan da özellikleri elde edebiliriz.

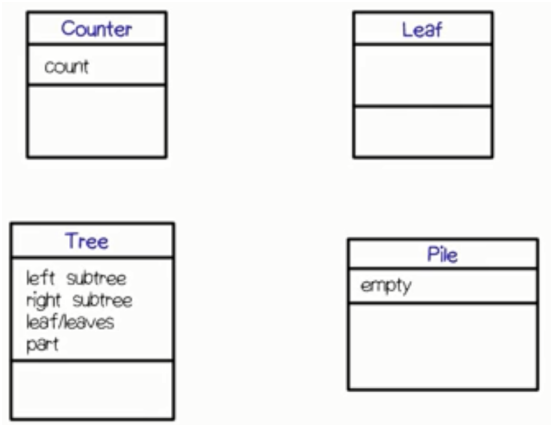
- Ağacın parçaları
- Yaprakların sayısı

Tek yaprak, yaprağın alabileceği sayı değerlerinden sadece biridir. Öyleyse, değeri bir olan bir sayı özelliğimiz olmalı.

İki alt ağaç, bir ağaçta olabilecek alt ağaçların sayısı özelliğinin bir değeri olmalı.

Parça, ağacın bir parçası olduğu için özellik olmalı.

Güncellenmiş Sınıf modelimiz



Bir ağacın yapraklarını saymak 4. Fiillerden işlemlere

İşlem: Nesnenin sağladığı bilişimsel servis.

- Eylem fiilleri ile bulunabilirler. Eylemleri ilgili olduğu sınıflara atarız.
  - İlgi fiilleri özellik belirtir. Ör: büyük görünen ev
  - Durum fiilleri nesne değil durum bildirir, nesneler arası ilişkileri oluşturur

Sayılmamış ağaç parçalarını boş yığına **koy**un.

Yaprakları sayacını sıfır **yap**ın.

Yığın boş olmadığı sürece, tekrar tekrar ağaç **al**ıp **incele**yin.

Ağaçta tek yaprak **var**sa, yaprakların sayacını **arttır**ın ve ağacı **at**ın.

İki tane alt ağaç varsa sol ve sağı **ayır**ıp ikisini de yığına **ekle**yin.

Yığın boşaldığında, yaprak sayısını **göster**in.

İşlemlerin ilgilileri kim?

Örneğimizde yığını kim tutacak?

- Tasarladığınız her sistemde, sistem de bir sınıf olabilir. Onun da sorumlulukları var.

Sayma işini kim yapacak? Neyi sayacak? * "**Sayılmamış**" kelimesi, aslında eylem değil durum belirtiyor

- İşlem ve argümanları
- Sayaç, yaprakları.

Ağacı (parçaları) yığına kim koyacak/yığından kim alacak?

- Yığının sorumluluğu, ağaç da argümanı

Kim Sıfır yapacak? Kim arttıracak?

- Sayaç

Kim inceleyecek?

- Sistem mi? Yığın mı?

  Şu aşamada statik yapıyı, sistemi anlamaya çalışıyoruz, davranış çözümlemesi daha sonra.

Ağacı kim atacak?

- Sayaç mı? Yığın mı?

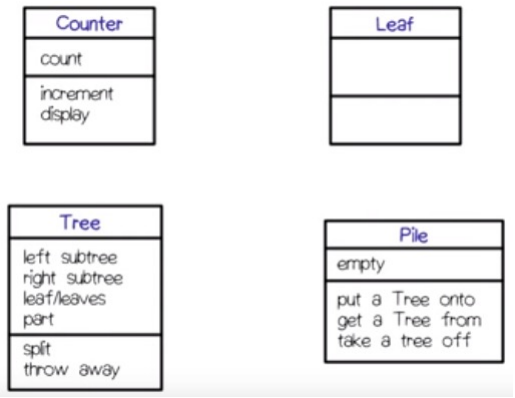
Ağacı kim bölecek?

Sınıf Modelimiz

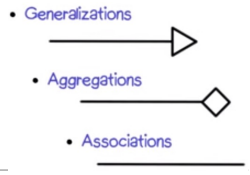
İşlemlerimizin parametrelerini, veri tiplerini, dönüş değerlerini de istersek belirtebilirdik.

Ancak esas amacımız,
- Problemin parçaları nelerdir ve
- Bunlar hangi özelliklerle temsil edilir
- Hangi servisleri sağlayacaktır

cevaplarını bulmaktır.

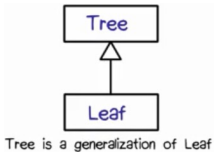


5. İlişkiler/Bağlantılar

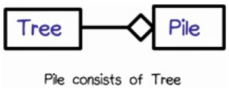


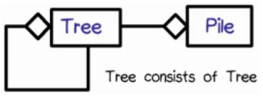
Sınıf modelinin sınıflardan geriye kalan yarısı

Genelleme (Generalization[specialization])
- Alt sınıfın nesneleri, üst sınıfın türündedir.
- Alt sınıf, üst sınıfın alt kümesidir.
- Aslında içindeydi, biz onu dışarı çıkarıp ayırdık
- Ör: Araba / Araç, Ağaç / Yaprak



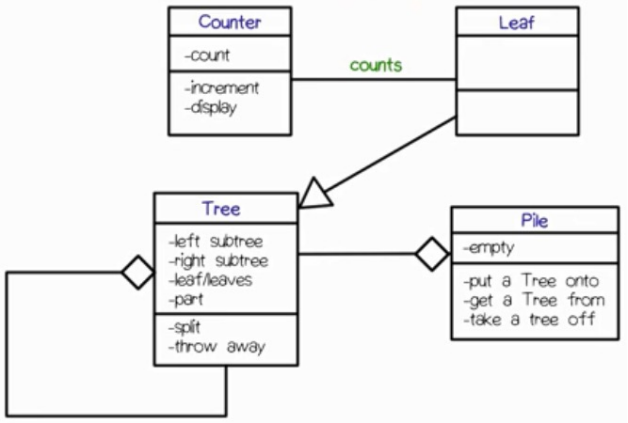
Birleşme (Aggregation) (Parça-bütün)
- Bir çeşit "X koleksiyonu", "X grubu"
- Içerir, aittir, kapsar, vardır, sahibidir, …





İşbirliği (Association)
- Durum bildiren fiillerle ortaya çıkar
- Bağlantının üzerine ismi yazılır
- Ör: Öğrenci üniversitede okur.
- Sayaç-Yaprak

Sınıf Modelimiz



Analiz modeli için uyarılar

Tüm sınıflarımız aslında herşeyi kapsayan AğaçYaprakSaymaSistemi sınıfındadır.

Genellikle diyagramlarda sistem sınıfı gösterilmez.

Normalde tipik gereksinim dokümanları yüzlerce sayfa olabilir ve pek çok özel kelime içerir.

Örneğimiz çok kısaydı ve aslında algoritma ve gerçekleştirme detayları içeriyordu.

Gereksinim dokümanlarında çözüm yolları yer almaz, sadece problemi içerirler.